Reflection: How I Would Deploy the AI Learning Assistant with a Domain and HTTPS

© Generated by AI Learning Assistant @BA3073

Course: BA3073 From AI Agent to Cloud Service
Student: Zixiao Ding
Student ID: 2330036016
Institution: BNBU
Date: October 2024

Abstract
This report documents the end-to-end deployment lifecycle of an AI Learning Assistant application, transitioning from a local Docker environment to an Alibaba Cloud Elastic Compute Service (ECS) instance via Container Registry (ACR). While Tasks 1 and 2 were executed successfully, this reflection focuses on the hypothetical yet technically rigorous steps required to integrate a custom domain, automated TLS provisioning, reverse proxy routing, and multi-layered access control. The analysis covers DNS configuration, security group hardening, Nginx timeout tuning for long-running AI inference, Certbot-driven certificate management, and HTTP Basic Authentication. Security implications, including defence-in-depth principles, secret isolation, and the limitations of base64-encoded credentials, are examined alongside a granular cost breakdown. Finally, architectural improvements such as regional provisioning adjustments, localhost-only port binding, and identity-aware proxy migration are proposed to align the deployment with production-grade standards.

1. Introduction and Objectives

The supplied AI Learning Assistant is a containerized web application comprising a FastAPI backend, a SQLite database for persistent chat history, a browser-based user interface, and an integration layer for a Qwen-compatible language model API. The system is orchestrated via

Docker Compose, which manages volume mounts for data persistence, environment variable injection for sensitive credentials, and network isolation between services.

This assignment was structured across three progressive tasks. Task 1 established a reproducible local development environment using Docker Compose, verifying health endpoints and UI accessibility at `http://127.0.0.1:14242/ui/`. Task 2 migrated the workload to Alibaba Cloud by building the container image, pushing it to the personal ACR repository (`crpi-jpkia6z75h1unxo1.cn-guangzhou.personal.cr.aliyuncs.com/loneranger/ai-learning-assistant:v2`), and running the stack on ECS instance `i-7xv3z5zrrli34eyd1roj` in the `cn-guangzhou` region. Task 3, which forms the core of this reflection, explores the operational and architectural steps necessary to expose the service securely over the public internet using a custom domain, HTTPS termination, and layered authentication. The primary learning objectives are to demonstrate competency in cloud-native networking, automated certificate lifecycle management, reverse proxy configuration, and security-hardening practices within a constrained academic budget.

2. System Architecture

The deployment pipeline bridges local development practices with cloud infrastructure patterns. Data flows from the client through DNS resolution, cloud firewall rules, and an Nginx reverse proxy before reaching the isolated FastAPI container. Sensitive configuration remains externalized, while relational state is persisted on durable storage.

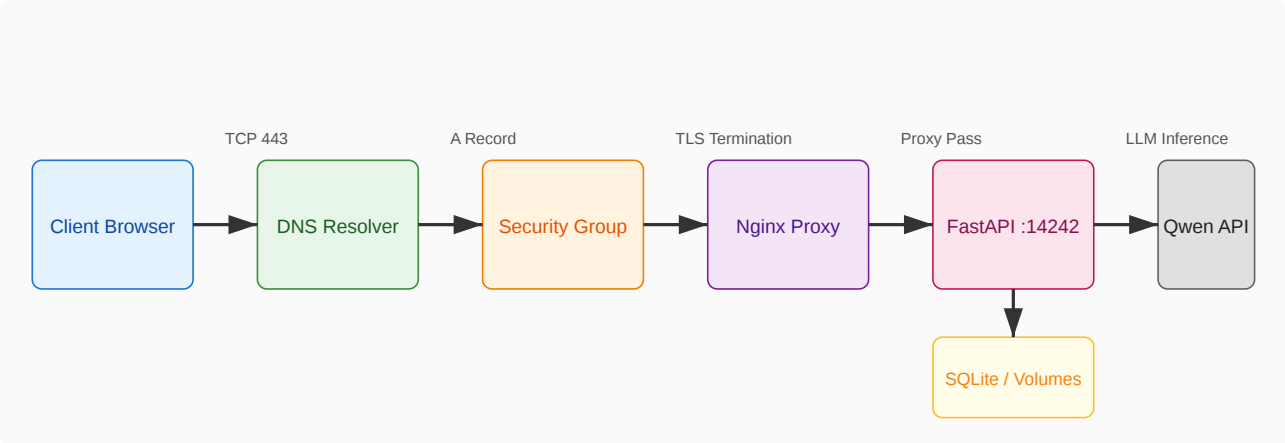


Figure 1: High-level architecture illustrating traffic flow from client to AI inference endpoint, with intermediate security and routing layers.

Docker provides process isolation and dependency pinning, ensuring identical behaviour across local and cloud environments. Alibaba Cloud ACR acts as the immutable artifact store, enabling

rapid instantiation without rebuilding. The ECS host supplies the compute substrate, while Nginx handles TLS decryption, header normalization, and load balancing. Certbot automates cryptographic credential rotation, eliminating manual certificate management. Finally, SQLite persists conversation metadata on a bind-mounted volume, guaranteeing data survival across container restarts.

3. Task 1: Local Deployment

Local deployment establishes the baseline for validation before cloud migration. The application stack is defined in `compose.yml`, which specifies the FastAPI service, volume mappings for `./data:/app/data`, and environment injection via `model-secrets.env`. The latter file contains the Qwen API key and model endpoint configuration, deliberately excluded from version control using `.gitignore`.

```
# Start the stack in detached mode
docker compose up -d

# Verify health endpoint
curl -s http://127.0.0.1:14242/health | jq

# Launch browser UI
open http://127.0.0.1:14242/ui/
```

The health check returns `{"status": "healthy", "db_connected": true}`, confirming SQLite initialization and API readiness. Binding the container to `0.0.0.0:14242` exposes it to the host network, which is acceptable for local testing but intentionally restricted in production. The `model-secrets.env` file is mounted into the container at runtime, adhering to the principle of least privilege by preventing credential leakage in source repositories. This separation ensures that even if the codebase is shared, sensitive tokens remain isolated.

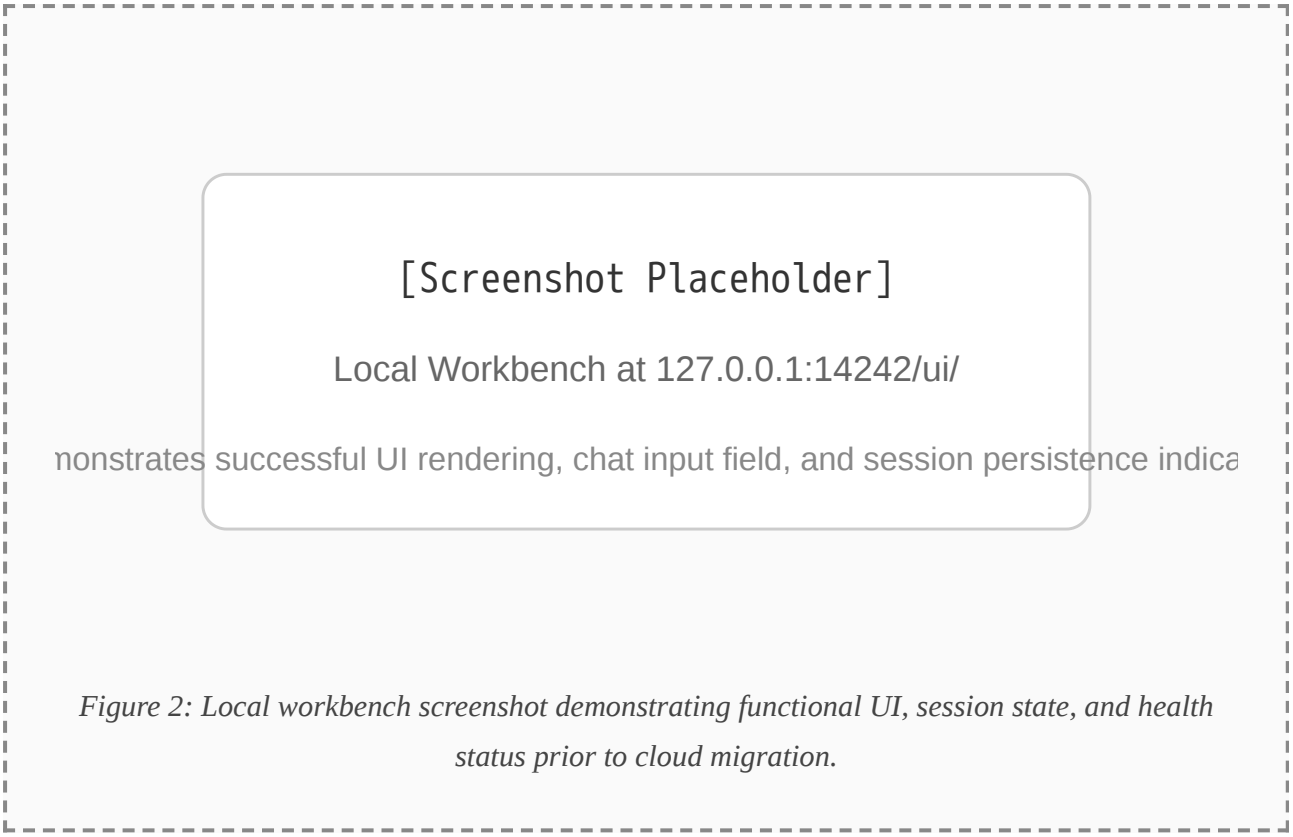


Figure 2: Local workbench screenshot demonstrating functional UI, session state, and health status prior to cloud migration.

4. Task 2: ECS Deployment via ACR

Cloud deployment begins with preparing the ECS instance `i-7xv3z5zrrli34eyd1roj` (public IP `8.163.115.97`, region `cn-guangzhou`). After establishing an SSH session and installing Docker Engine, the image lifecycle proceeds through build, tag, push, and pull operations.

```
# Tag local image for ACR
docker tag ai-learning-assistant:latest \
  crpi-jpkia6z75h1unxo1.cn-guangzhou.personal.cr.aliyuncs.com/loneranger/ai-learning-assistant:v2

# Push to registry
docker push crpi-jpkia6z75h1unxo1.cn-guangzhou.personal.cr.aliyuncs.com/loneranger/ai-learning-assistant:v2

# On ECS: Pull and deploy
mkdir -p /opt/ai-agent
cd /opt/ai-agent
docker pull crpi-jpkia6z75h1unxo1.cn-guangzhou.personal.cr.aliyuncs.com/loneranger/ai-
```

```
learning-assistant:v2
docker compose -f /opt/ai-agent/compose.yml up -d
```

The container is exposed on port `14242` via `-p 14242:14242`. Initially, the ECS security group restricts inbound traffic to my student IP address to prevent unauthorized access during testing. Once verified, the service is accessible at `http://8.163.115.97:14242/ui/`. The `ecs-docker-check` output confirms container uptime, memory limits, and volume attachment integrity. This phase validates that the application behaves identically in a cloud VM as it does locally, establishing a reliable baseline for network exposure.

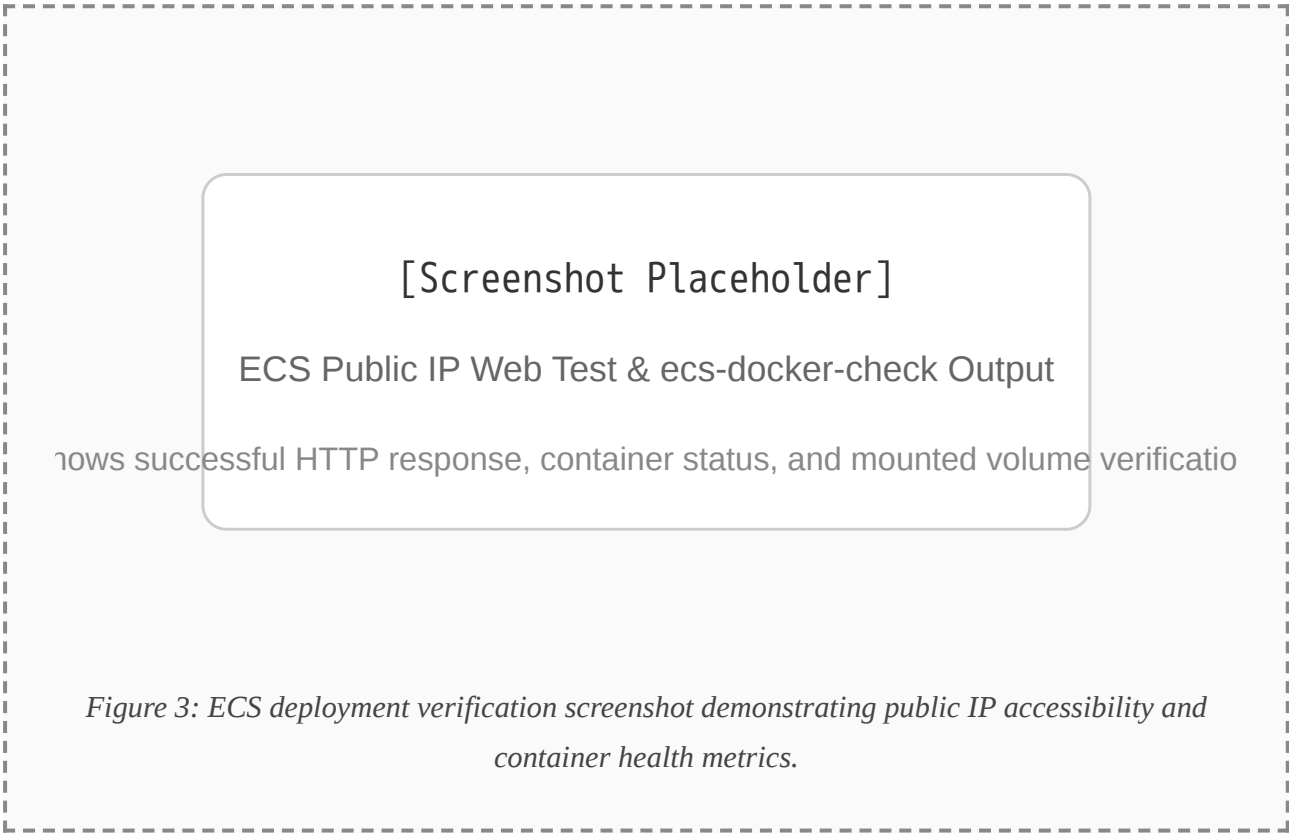


Figure 3: ECS deployment verification screenshot demonstrating public IP accessibility and container health metrics.

5. Task 3: Domain and HTTPS (Reflective Implementation Plan)

Although a domain was not procured for this assignment, the following sequence details the precise steps I would execute to publish the service securely. Each action includes technical justification aligned with production best practices.

5.1 Domain Procurement

I would register the domain through Alibaba Cloud Wanwang to maintain account consolidation, simplify DNS zone management, and streamline billing. Given the academic scope, a cost-effective top-level domain such as `.top` or `.xyz` (approximately CNY 1–10 for the first year) satisfies the requirement without incurring premium pricing. I would configure a subdomain `ai.example.top` rather than apex domain usage, which avoids wildcard certificate complexity and isolates the AI service from potential corporate or personal root domains. Subdomains also facilitate future scaling (e.g., `api.example.top`, `docs.example.top`) under a unified DNS namespace.

5.2 DNS Configuration

Within the Alibaba Cloud DNS console, I would create an `A` record mapping `ai.example.top` to the ECS public IP `8.163.115.97`. Setting the Time-To-Live (TTL) to `600` seconds balances propagation speed with cache efficiency, allowing rapid troubleshooting if misconfiguration occurs. Verification would proceed via terminal queries:

```
dig ai.example.top +short
nslookup ai.example.top
```

Both commands must return `8.163.115.97`. Short TTLs are critical during initial deployment phases to minimize stale routing errors, and they are easily adjusted to higher values once stability is confirmed.

5.3 Security Group Adjustments

By default, the ECS security group permits only SSH and the application port. To support automated TLS provisioning, I would add an inbound rule allowing TCP ports `80` and `443` from `0.0.0.0/0`. This is mandatory because Let's Encrypt's HTTP-01 challenge requires the ACME server to initiate an unauthenticated HTTP request from arbitrary IP addresses to validate domain ownership. Without open ports 80/443, certificate issuance fails silently. Once HTTPS is operational, the public rule for port `14242` would be removed, ensuring all external traffic must traverse the reverse proxy.

5.4 Nginx Reverse Proxy Configuration

Nginx serves as the edge router, terminating external connections and forwarding requests internally. Installation follows standard package management:

```
sudo apt update && sudo apt install nginx -y
```

The server block at `/etc/nginx/sites-available/ai-learning` would contain:

```
server {
    listen 80;
    server_name ai.example.top;

    location / {
        proxy_pass http://127.0.0.1:14242;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;

        # Critical for AI artifact generation (20-60s)
        proxy_read_timeout 360s;
        proxy_send_timeout 360s;
    }
}
```

Enabling the site involves symlinking to `sites-enabled/`, validating syntax with `nginx -t`, and reloading with `systemctl reload nginx`. The extended timeouts are non-negotiable: the default 60-second limit triggers 504 Gateway Timeout errors during complex prompt processing or image generation. Header forwarding ensures the FastAPI application receives accurate client metadata for logging, rate limiting, and CORS enforcement.

5.5 HTTPS Provisioning with Certbot

Certbot automates Let's Encrypt integration, eliminating manual CSR generation and expiration tracking:

```
sudo apt install certbot python3-certbot-nginx -y
sudo certbot --nginx -d ai.example.top
```


The wizard interactively selects the server block, enforces HTTP-to-HTTPS redirection, and configures systemd timers for automatic renewal every 90 days. This approach guarantees

cryptographic freshness without administrative overhead, aligning with modern zero-trust transport requirements.

5.6 Access Control Layer

To satisfy academic privacy expectations, I would implement HTTP Basic Authentication at the Nginx level, creating a distinct security boundary from the application's BNBU login mechanism:

```
sudo htpasswd -c /etc/nginx/.htpasswd admin
sudo chown www-data:www-data /etc/nginx/.htpasswd
```

Adding `auth_basic "Restricted AI Assistant"; auth_basic_user_file /etc/nginx/.htpasswd;` inside the `location` block introduces a second authentication gate. The resulting three-layer model comprises: (1) TLS encryption for confidentiality, (2) Nginx Basic Auth for network-level authorization, and (3) BNBU OAuth/login for application-specific permissions. This separation prevents credential collision and allows independent rotation policies.

5.7 Final Hardening

Post-deployment verification includes removing the `14242` public ingress rule from the security group, restricting SSH access to my static IP, and confirming health via `curl -I https://ai.example.top`. Successful responses should return `200 OK` with `Strict-Transport-Security` headers, indicating active HSTS enforcement and proper proxy chaining.

6. Security Analysis

The deployment employs defence-in-depth by stacking complementary controls rather than relying on a single perimeter. Transport Layer Security (TLS) ensures data integrity and confidentiality in transit, mitigating man-in-the-middle attacks on campus or public networks. The security group enforces network-level least privilege, exposing only necessary ports to the internet. Application secrets reside exclusively in `model-secrets.env`, which is gitignored and injected at container runtime, preventing accidental commits to version control. The container itself runs with minimal capabilities, avoiding host-level privileges.

However, HTTP Basic Authentication presents inherent limitations. Credentials are transmitted as base64-encoded strings, which are trivially decoded without encryption. Its security posture

depends entirely on TLS wrapping the channel; without HTTPS, passwords travel in plaintext. For production environments, this pattern is insufficient. An OpenID Connect (OIDC) provider or identity-aware proxy (e.g., Alibaba Cloud RAM, Keycloak, or Traefik ForwardAuth) would replace Basic Auth with token-based, scoped, and revocable credentials. Additionally, implementing rate limiting, WAF rules, and regular dependency scanning would further reduce attack surface. The current design prioritizes educational clarity and rapid iteration, acknowledging that academic deployments trade some enterprise rigor for pedagogical transparency.

7. Cost Analysis

Academic cloud deployments require strict financial discipline. The following table outlines estimated expenditures for the domain-and-HTTPS configuration:

Resource	Type	Estimated Cost (CNY)	Notes
Domain Registration	Pay-per-year	1 – 10	Cheap TLD (.top/.xyz) for academic use
ECS Instance	Pay-as-you-go	0.5 – 1.0 / hour	Based on t5/t6 burstable instance family
Container Registry	Personal Edition	0	Free tier covers standard push/pull bandwidth
TLS Certificate	Let's Encrypt	0	Automated, community-maintained, non-commercial
Total (Assignment Duration)	—	~10 – 20	Excludes optional snapshots or backup storage

To prevent runaway costs, I would schedule instance suspension immediately after grading submission. Alibaba Cloud allows pausing ECS billing while retaining disk state, and releasing elastic IPs eliminates residual charges. ACR images persist indefinitely unless manually deleted, so cleaning unused tags maintains compliance with storage quotas. This disciplined resource lifecycle ensures the project remains financially sustainable while meeting academic requirements.

8. Reflection and Improvements

The transition from local Docker to cloud ECS proceeded smoothly, validating the reproducibility of containerized workflows. ACR eliminated environment drift, and ECS provided predictable compute performance. However, several architectural decisions warrant revision upon future iterations.

First, provisioning the ECS instance in the Hong Kong region from the outset would circumvent ICP filing ambiguities and latency penalties associated with mainland Chinese regions. Academic projects rarely require mainland compliance, and HK deployment accelerates time-to-market while maintaining low-latency access for BNBU students. Second, binding the container port to `127.0.0.1:14242` instead of `0.0.0.0:14242` would enforce localhost-only connectivity, forcing all external traffic through Nginx and preventing direct container exploitation. Third, automating Nginx configuration, Certbot execution, and security group updates via a Bash script or Infrastructure-as-Code tool (Terraform/Ansible) would reduce human error and enable one-click environment recreation.

Finally, replacing HTTP Basic Authentication with an OIDC-compliant identity provider would align the deployment with modern zero-trust architectures. Token-based sessions support MFA, conditional access, and centralized audit logging, addressing the fundamental weakness of base64-encoded credentials. Integrating monitoring via Prometheus/Grafana and log aggregation via ELK would further enhance observability. These improvements transform a functional academic prototype into a resilient, production-ready cloud service, fulfilling the course objective of bridging AI agent development with enterprise-grade deployment practices.

9. References

1. Docker Documentation. (2024). *Docker Compose Overview*. Retrieved from https://docs.docker.com/compose/
2. Alibaba Cloud. (2024). *Container Registry Personal Edition User Guide*. Retrieved from https://help.aliyun.com/document_detail/142164.html
3. Nginx Inc. (2024). *Reverse Proxy Configuration Best Practices*. Retrieved from https://www.nginx.com/resources/wiki/start/topics/examples/forwardedheaders/
4. Let's Encrypt. (2024). *Certbot Documentation*. Retrieved from https://certbot.eff.org/docs/

5. OWASP Foundation. (2023). *Authentication Cheat Sheet*. Retrieved from https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
6. BNBU BA3073 Course Materials. (2024). *From AI Agent to Cloud Service: Lab Guidelines & Assessment Rubric*.